

Assignment 2

1. A junior developer writes the following code and complains it "doesn't work right." Explain what is wrong and why, then rewrite it correctly:

```
const user = {
  username: "tolu",
  getUsername: () => {
    return this.username;
  }
};
console.log(user.getUsername());
```

Answer

It doesn't work right because an arrow function was assigned as a value to the key getUsername instead of a regular unnamed function. This happened because arrow functions do not adhere to the rules of scope. When used with the "this" keyword, it returns as undefined.

```
const user = {
  username: "tolu",
  getUsername: function () {
    return this.username;
  },
};
console.log(user.getUsername());
```

2. Write a createWallet function that tracks a private balance using closure. It should return an object with three methods: credit(amount), debit(amount), and getBalance(). The debit method should throw an error if the amount exceeds the current balance. The balance itself should not be directly accessible from outside the function.

Answer

```
function createWallet() {
  let balance = 0;
  return {
    credit(amount) {
      balance += amount;
    },
    debit(amount) {
      if (balance < amount) throw new Error("Insufficient Funds");
      balance -= amount;
    },
  };
}
```

```

    getBalance() {
        return balance;
    },
};
}

const myAccount = new createWallet();
myAccount.credit(5000);
console.log(myAccount.getBalance());
myAccount.debit(300);
console.log(myAccount.getBalance());

```

3. Write a class Subscription that:

- Accepts a plan (e.g. "basic" or "premium") and a pricePerMonth in its constructor
- Has a private field #active defaulting to false
- Has activate() and cancel() methods that toggle #active
- Has a getter monthlyCost that returns 0 if the subscription is inactive, otherwise returns pricePerMonth

Answer

```

class Subscription {
    #active;
    constructor(plan, pricePM) {
        this.plan = plan;
        this.pricePM = pricePM;
        this.#active = false;
    }

    activate() {
        this.#active = true;
    }

    cancel() {
        this.#active = false;
    }

    get monthlyCost() {
        if (this.#active == false) {
            return 0;
        }
        return this.pricePM;
    }
}

```

```
const mySubscription = new Subscription("Basic", 2500);
mySubscription.activate();
console.log(mySubscription.monthlyCost);
mySubscription.cancel();
console.log(mySubscription.monthlyCost);
```

4. What will the following code output and why? Name the OOP concept it demonstrates:

```
class Notification {
  send() { return "Sending generic notification"; }
}
class EmailNotification extends Notification {
  send() { return "Sending email"; }
}
class SMSNotification extends Notification {
  send() { return "Sending SMS"; }
}
const alerts = [new EmailNotification(), new SMSNotification()];
alerts.forEach(a => console.log(a.send()));
```

Answer

Output: Sending email
Sending SMS

This demonstrates the Polymorphism concept in OOP. This is because polymorphism allows you to use the same method/class but have different outcomes. In the case of the example above, both classes inherited the Notification class and the send method in it. In the case of the EmailNotification class, the send method was overridden to return "Sending email" instead of "Sending generic notification", while SMSNotification returned "Sending SMS" instead of "Sending generic notification". Basically, one class gave birth to two different forms.

5. Explain the difference between Object.keys(), Object.values(), and Object.entries(). Write a function summarize(obj) that accepts any object and logs each key alongside its value in the format: "key → value".

Answer

```
let Ade = {
  lastname: "Somolu",
  age: 22,
  gender: "male",
};
```

Object.keys(): returns an array of the keys in the key-value pairs of all the properties/attributes in an Object.

```
const keys = Object.keys(Ade);
```

```
console.log(keys);
```

Output: ['lastname', 'age', 'gender']

Object.values(): returns an array of the values in the key-value pairs of all the properties/attributes in an Object.

```
const values = Object.values(Ade);
```

```
console.log(values);
```

Output: ['Somolu', 22, 'male']

Object.entries(): returns an array of arrays of the key-value pairs of all the properties/attributes in an Object. The key-value pair is separated by a comma instead of a column and enclosed in a square bracket.

```
const myEntries = Object.entries(Ade);
```

```
console.log(myEntries);
```

Output: [['lastname', 'Somolu'], ['age', 22], ['gender', 'male']]

//Summarize Function.

```
function summarize(obj) {
```

```
  for (let key in obj) {
```

```
    console.log(`${key} → ${obj[key]}`);
```

```
  }
```

```
}
```

```
summarize(Ade);
```

6. You join a team and clone a Node.js project from GitHub. The node_modules folder is not included. What command do you run to get the project working, and why does that command know what to install?

Answer

The command to run to get the project working is "npm install". This command looks into the package.json file in the project and directly downloads all the dependencies listed in it and their exact versions. This makes sure that what is needed to function properly is what is being downloaded.

7. A package.json file has this entry: "axios": "~1.4.0". Another project has "axios": "^1.4.0". What is the difference between what each will install? Which would you recommend for a production app and why?

Answer

The difference between the first package.json and the second package.json is that the first project can only work with versions of axios ranging from 1.4.0 to less than 1.5.0. It can only tolerate **patch updates/bug fixes** and not a **major** or **minor** change, while the second project can work with versions of axios that have had a **minor (added functionality/feature)** and/or **patch update (bug fixes)**, i.e 1.4.0 to less than 1.5.0.

For optimum security, I would recommend that "axios": "~1.4.0" be used. This version makes sure that only bug-fixing updates are carried out, therefore eliminating the potential upgrade that could break the application/software. i.e. no MINOR and MAJOR updates, only PATCH.

8. You need to write a Node.js script that reads a file path from the user and prints just the filename (without the directory). Which built-in Node.js module would you use? Write the code to do it.

Answer

The built-in modules I would use are **readline** (to enable the terminal to receive input from the user and display the resulting output) and **path** (to work with the file path entered by the user)

```
const path = require("path");
const readline = require("readline");

const script = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

script.question("Enter Path name ", (selected) => {
  console.log(`The name of the File is ${path.basename(selected)}`);
  script.close();
});
```

9. A teammate created a custom module tax.js that exports a function calculateTax(price, rate). Write the code in index.js to import and use it using CommonJS. Then rewrite the same thing using ES Modules.

Answer

For **CJS**

```
// index.js
const tax = require("./my_modules/tax");
let currentTax = tax.calculateTax(1000, 3.5);
console.log(currentTax);

// tax.js
function calculateTax(price, rate) {
  return price * rate;
}

module.exports = { calculateTax };
```

For **ESM**

```
// index.js
```

```
import { calculateTax } from "./my_modules/tax.js";
let currentTax = calculateTax(1000, 3.5);
console.log(currentTax);

// tax.js
export function calculateTax(price, rate) {
  return price * rate;
}
```

10. Explain two reasons why ES Modules are preferred over CommonJS for modern Node.js projects. What is the one package.json change required to opt in?

Answer

ESM is preferred over CommonJS for modern Node.js projects because:

- ESM enforces module scope; variables and attributes in a module are restricted to that module and cannot be accessed globally. They can only be accessed through the importation of that module. This ensures security and distinction of the code.
- ESM practices a static structure, which means that imports and exports are determined at load time rather than at runtime. This ensures that only the required code is loaded and that errors are pointed out before runtime, which in turn leads to faster loading.

The required package.json change is:

"type": "commonjs" to "type": "module"

11. Write an ESM module currency.js that exports a default function convert(amount, rate) and a named export SUPPORTED_CURRENCIES (an array of currency codes). Show how to import both in another file.

Answer

```
// index.js
import myDefaultFunction, {
  SUPPORTED_CURRENCIES,
} from "./my_modules/currency.js";
console.log(SUPPORTED_CURRENCIES);
let converted = myDefaultFunction(100, 1366.66);
console.log(converted);

// currency.js
export default function convert(amount, rate) {
  return amount * rate;
}

export const SUPPORTED_CURRENCIES = [
  "USD", "EUR", "GBP", "CAD", "NGN", "AED", "GHS"];

```